VIDEO NOTES · PART 10

Runnables and LCEL

The building blocks everything else is made from

What this document is

A plain-English guide to runnables — the shared interface that lets LangChain components snap together.

1 The LEGO Brick Idea

A **runnable** is anything that takes an input and returns an output through the same standard interface.

That sounds almost too simple to matter. It matters enormously. Because prompts, models, parsers and entire chains all follow the same interface, any of them can connect to any other without adapter code in between.

Figure 1 — Identical connectors on every piece. That is the whole idea.

The LEGO comparison is exact. Bricks fit together not because they are similar in size or purpose, but because they share the same studs. Runnables are the studs.

> **Why this is the most important component to understand:** chains, memory and RAG pipelines are all built from runnables. Once the interface makes sense, the rest of LangChain stops feeling like a collection of unrelated classes.

2 LCEL and the Pipe Operator

LCEL — the LangChain Expression Language — is the notation for joining runnables together. It uses a single symbol: the pipe, written |.

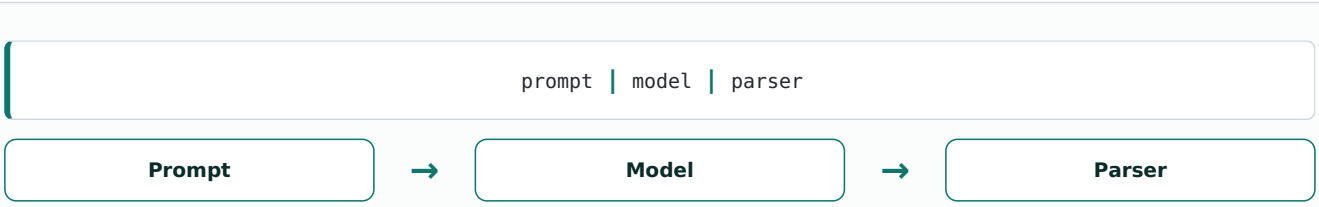

Figure 2 — The pipe reads left to right: each component's output becomes the next one's input.

Anyone who has used a shell command line will recognise the idea immediately. The pipe sends what came out of one thing into the next thing.

What this replaces

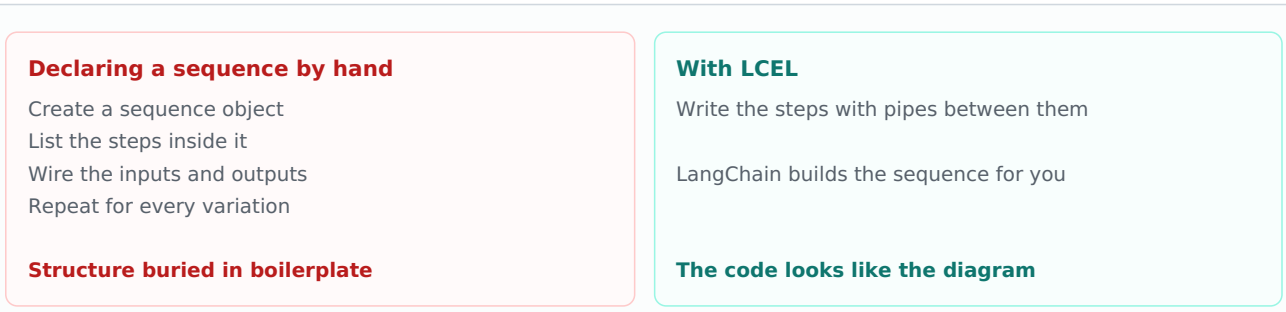

Figure 3 — A runnable sequence forms automatically; you do not declare one explicitly.

> **The readability argument is the real one.** When the code reads in the same order the data flows, a reviewer can see the whole pipeline at a glance instead of tracing variables through it.

3 The Five Runnable Types

Beyond the basic sequence, LangChain provides runnables for the shapes a real pipeline needs.

Runnable	What it does
RunnableSequence	Runs steps one after another — what the pipe creates
RunnableParallel	Runs several branches at once on the same input
RunnableLambda	Turns any Python function into a runnable
RunnableBranch	Chooses a path based on a condition
RunnablePassthrough	Carries the input forward unchanged

RunnableParallel — several branches at once

Give it the same input and it runs every branch simultaneously, returning the results together as a dictionary with one key per branch.

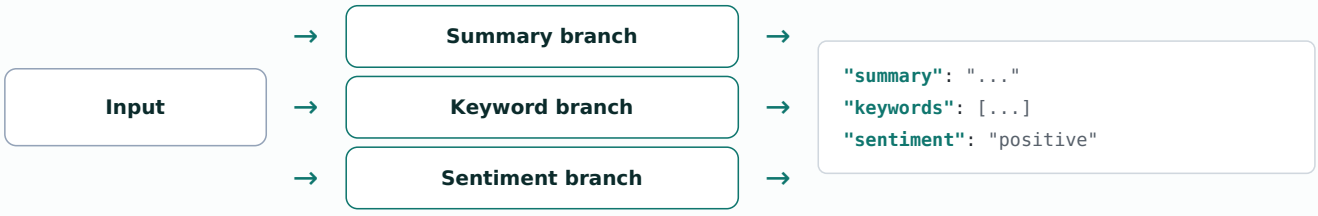

Figure 4 — You name the branches; those names become the keys in the result.

> **The dictionary shape is the useful part.** Because results come back under names you chose, the next step in the pipeline can pick out exactly the piece it needs rather than unpacking a positional list.

RunnableLambda — your own code as a step

Not every step needs a model. Sometimes you need to clean a string, reformat a date, filter a list, or call your own database function.

RunnableLambda wraps an ordinary Python function so it becomes a runnable, which means it can sit in a pipeline alongside prompts and models.

Figure 5 — Custom logic drops into the middle of a pipeline with no special handling.

> **This is the escape hatch that keeps LCEL practical.** Without it, anything the framework did not anticipate would force you out of the pipeline entirely. With it, the awkward business-specific step is just another link.

RunnableBranch — conditional paths

Pipelines are not always straight lines. **RunnableBranch** checks conditions in order and runs the first matching path — the pipeline equivalent of if / else if / else.

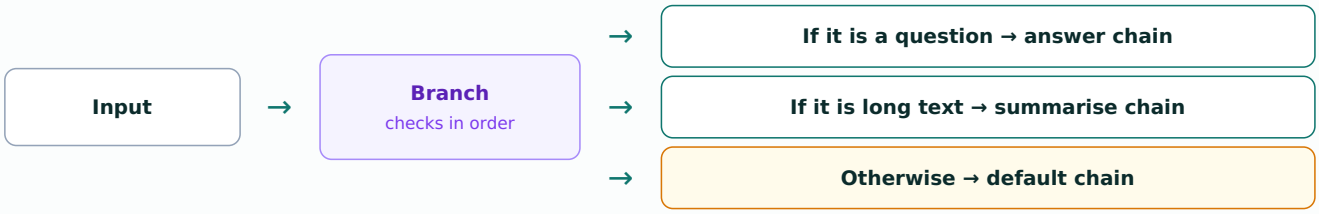

Figure 6 — Conditions are evaluated top to bottom; the first match wins and the rest are skipped.

Order matters, and the default is not optional. Because the first matching condition wins, a broad condition placed early will swallow inputs meant for a later, more specific branch. Always finish with a default so nothing falls through with nowhere to go.

Branch versus router

RunnableBranch	Router chain (Part 9)
Conditions you write in code — checks on the input itself, evaluated in order	Typically a model classifies the input first, then the result decides the path
Fast, free, completely predictable	Handles messy natural language that no simple condition could match

Prefer a branch where a simple rule works. If you can decide the path with a plain condition, do — it costs nothing and never misclassifies.

RunnablePassthrough — keeping the original input

Pipelines have a natural problem: each step replaces what came before it. Once a step transforms the input, the original is gone — but a later step often still needs it.

RunnablePassthrough solves this by carrying the input forward unchanged, usually inside a parallel block alongside the step that transforms it.

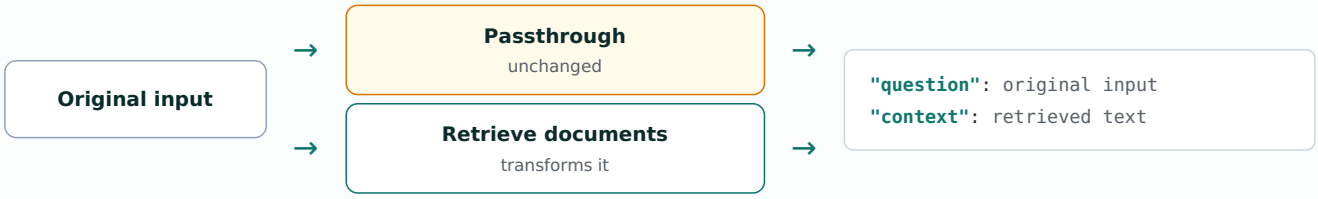

Figure 7 — Both the original question and the newly fetched context arrive at the next step together.

This is the standard RAG shape. To answer a question from your documents, the final prompt needs both the retrieved text *and* the question that was asked. Passthrough is how the question survives the retrieval step. Expect to see this pattern constantly in the RAG lectures ahead.

4 All Five Working Together

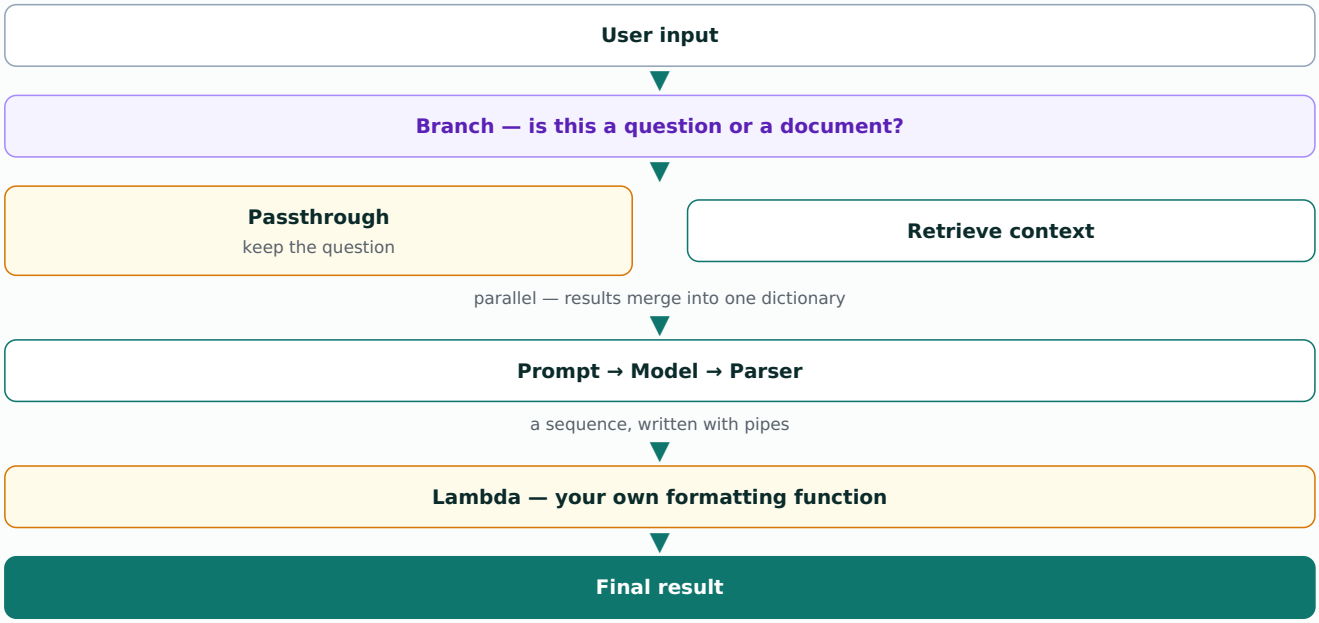

Figure 8 — One pipeline using every runnable type covered in this lecture.

Quick reference

Reach for	When you need to	Returns
Sequence (pipe)	Run steps in order	The last step's output
Parallel	Run branches on one input	A dictionary of named results
Lambda	Insert your own code	Whatever your function returns
Branch	Pick a path by condition	The matching branch's output
Passthrough	Keep the input for later	The input, unchanged

5 What Comes Next

The instructor lays out the plan from here. First, a **production-grade project** applying everything covered so far — prompts, parsers, memory, chains and runnables — before adding anything new.

After that, the course moves into **RAG**, starting with its fundamentals.

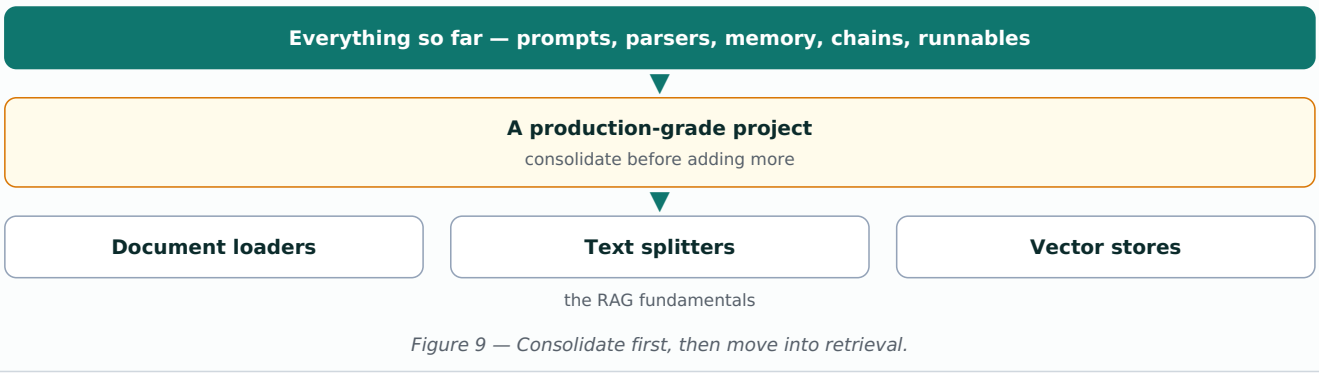

Figure 9 — Consolidate first, then move into retrieval.

> **Why the project comes before RAG:** building something end to end is what exposes the gaps. Concepts that felt

clear in isolation reveal their rough edges the moment they have to work together under real conditions.

Practice before the next lecture

- ☐ Rewrite an existing chain using pipes and compare how it reads.
- ☐ Build a parallel runnable with three named branches and inspect the dictionary it returns.
- ☐ Wrap a small Python function in a lambda and place it at the end of a pipeline.
- ☐ Write a branch with two conditions and a default, then send it an input matching neither.
- ☐ Use a passthrough so a later step can still see the original question.
- ☐ Combine all five into one pipeline and trace what each step receives.

The short version: a runnable is anything with the same input-output interface, LCEL joins them with the pipe operator, and the five types cover sequence, parallel branches, your own code, conditional paths, and carrying the original input forward.